Drive Launcher Blueprint

The same app, the same screens and the same idea, on a structure you can trust: one owner for the session, a lock that holds for as long as the work does, a sync that knows what changed on each side, and transfers that run in parallel and report their progress.

DRAWING	STATUS	BUILDS ON
Session, sync and UI	Proposed design	Part 1 · Drive Launcher As-Built (findings F1–F41)
DATE	**STACK**	**LEVEL**
14 October 2026	Unchanged: Electron, React, zustand, chokidar, plus Bottleneck	Junior-friendly: every new term is explained where it first appears

01 What we're building

The goals come straight from how you described the app:

- **Show which files changed during the session**, live, while people play.
- **Notice changes made between sessions**, on your PC or in the cloud, and warn before anything gets overwritten.
- **Move files in parallel lanes** that adapt to your bandwidth and to Google's rate limits, with each lane's current file, failures, an overall progress bar and a time estimate.
- **Have a structure** where adding the next feature doesn't break the last one.

Six rules shape everything below. They're the opposite of the patterns behind Part 1's findings.

One owner per fact

The main process owns all state. Each fact has exactly one module that writes it; the UI only displays what it's told.

Decide, then do

Pure functions decide what should happen and return a plan. Separate code carries the plan out. Deciding is easy to test; doing is easy to watch.

Name the phases

A session moves through named phases. Each button and each event is allowed only in certain phases, so repeats and races do nothing.

Hold the lock for the whole job

The lock is renewed from the moment you claim it until the upload is committed, and every renewal checks it's still yours.

Expect failure

Every network call can fail. Retry what can be retried, report the rest clearly, and never report success for work that didn't happen.

Type the edges

One shared contract describes every IPC channel. Errors cross it as values, never as surprises.

02 The new shape

Compare this with Figure 1 of Part 1. There, main.ts called everything and two services imported each other. Here the code is arranged in layers. **Arrows only point down**: a layer may use the ones below it, never the ones above. Information that has to travel up, such as "the upload is 40% done", travels as events.

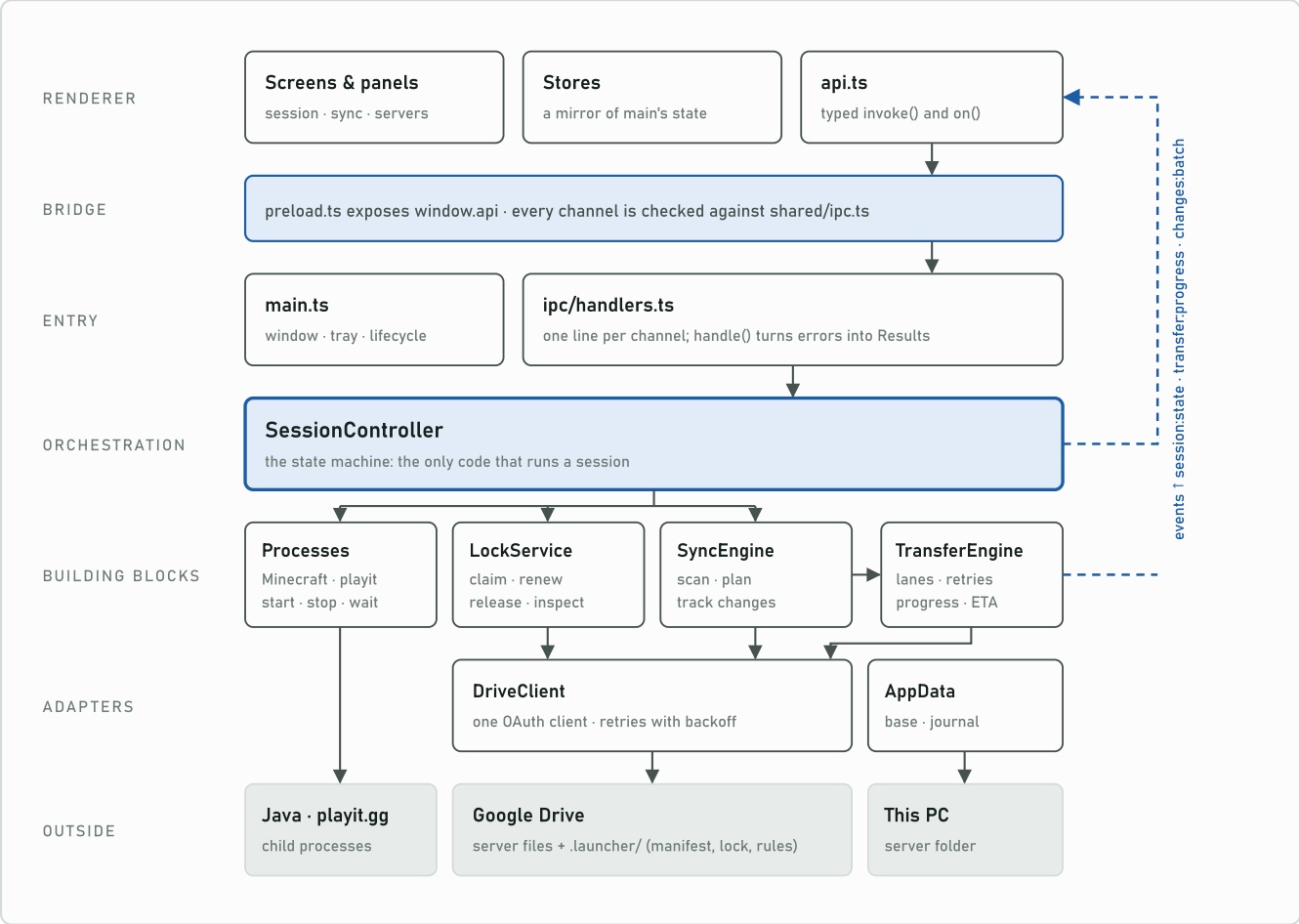


Figure 1. The proposed layers. Calls go down; progress and state come back up as events. SessionController is the only code that runs a session; SyncEngine uses TransferEngine to move files; nothing below the bridge knows the UI exists. (SyncEngine also reads and writes the base snapshot through AppData and scans the server folder directly; those arrows are left out to keep the picture readable.)

What each layer is for:

- **Renderer.** Draws the state it's given and sends intents ("start this server", "stop"). It never runs a sequence of steps itself.
- **Bridge.** A contract file both sides import, so a renamed channel breaks the build instead of the app.
- **Entry.** main.ts shrinks to window, tray and app lifecycle. Handlers are one-liners that call a service; the shared `handle()` wrapper converts errors into values.
- **Orchestration.** SessionController is the one place that knows the order of things: check, claim, download, launch, run, stop, upload, release.

- **Building blocks.** Each does one job. LockService doesn't know about transfers and TransferEngine doesn't know about locks; SyncEngine uses TransferEngine only through its `run()` method.
- **Adapters.** Thin wrappers around the outside world. DriveClient is the only code that talks HTTP to Google, so retries, token refresh and rate-limit handling live in exactly one place.

03 Where the code goes

Folders are named after responsibilities, not after kinds of code: no `utils`, no `helpers`. A good test: if you can't say what a file is for in one short line, it's doing two jobs.

```
shared/                      imported by both processes
  ipc.ts                     the contract: every channel, its arguments and its result
  types.ts                   Manifest, Snapshot, SyncPlan, SessionState, AppError
  paths.ts                   toPosix() and ignore rules: one path format everywhere
electron/
  main.ts                    window, tray, app lifecycle; creates and wires the services
  ipc/
    handle.ts                the one place where thrown errors become Results
    handlers.ts              one line per channel
  session/
    SessionController.ts     the state machine; the only code that runs a session
  lock/
    LockService.ts           claim, renew, release, inspect (read-only)
  sync/
    manifest.ts              read and write manifest.json (v2) and the local base
    scanner.ts               walk the folder, hash only what looks different
    planner.ts               planSync(base, local, remote): a pure function
    changeTracker.ts         chokidar, batched into one event per half second
  transfer/
    TransferEngine.ts        job queue on Bottleneck, retries, progress, ETA
    LaneController.ts        decides how many transfers run at once
  drive/
    DriveClient.ts           one OAuth client, token refresh, retry with backoff
    driveFiles.ts            get, create, update, download, delete, ensureFolders
  processes/
    MinecraftProcess.ts      start, stop and wait for exit, player list
    PlayitProcess.ts         start, read the address with a timeout, stop
  store/
    appData.ts               paths and JSON files in the app's data folder
src/
  api.ts                     typed invoke() and on() for components
  stores/                    sessionStore (mirror), serversStore, settingsStore
  features/
    session/                 Power button, progress line, console
    sync/                    SyncStatusCard, ChangesPanel, TransfersPanel
    servers/                 server picker, Drive Folders, sharing
    settings/                Scripts, ignore rules
```

Figure 2. Proposed layout. The renderer is grouped by feature so everything about one screen sits together; the main process is grouped by responsibility.

04 The session as a state machine

A **state machine** is a fancy name for a simple idea: the session is always in exactly one named phase, and only certain moves between phases are allowed. You already have the beginnings of

one in lock.json's status field. The difference is that the phase now lives in one object in the main process, and every action checks it first.

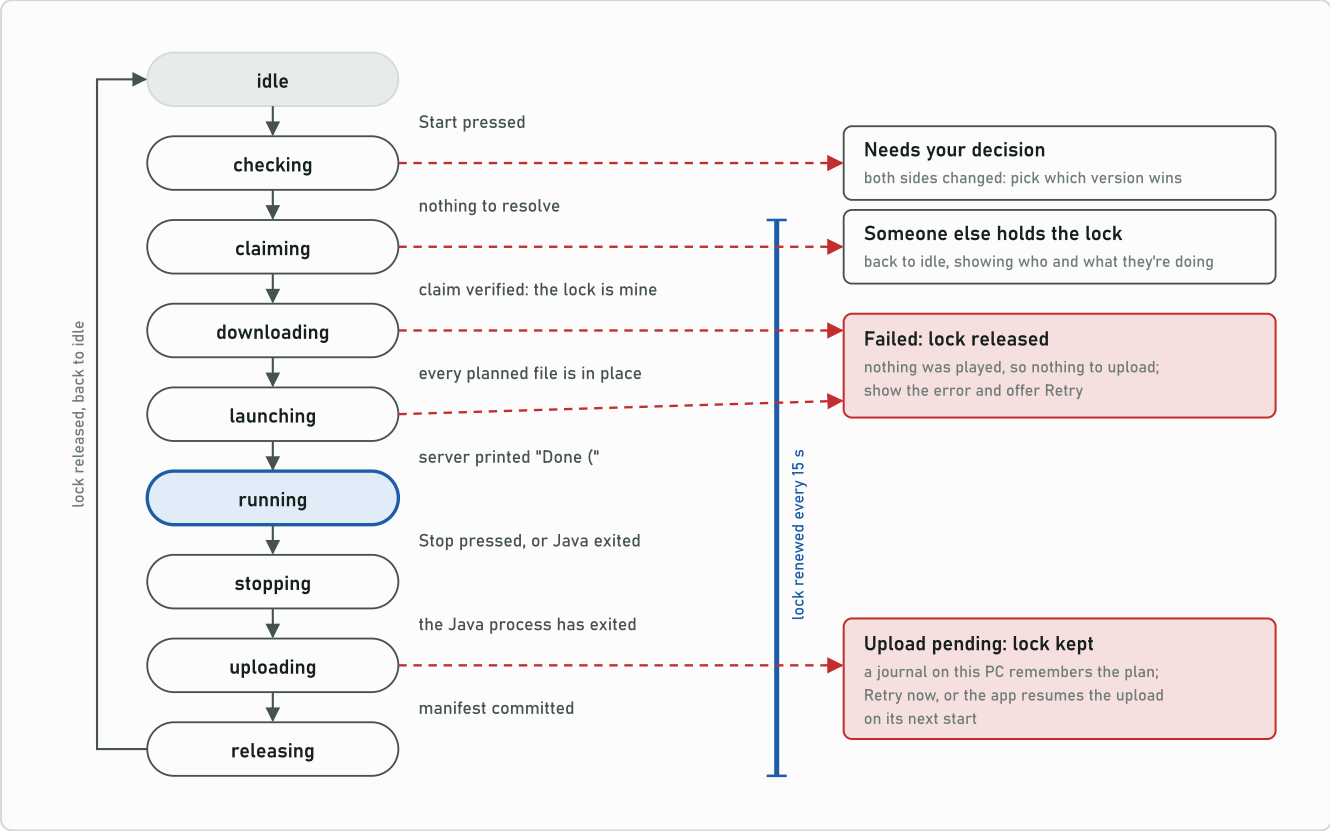


Figure 3. Session phases. The left column is the normal path; the boxes on the right are the ways out of it. The blue bar shows that the lock is held, and renewed, for the whole time the world is in motion, not just while the game runs.

PHASE	WHAT THE APP DOES	LOCK.JSON SAYS	WHAT THE USER SEES
idle	Nothing running; the preflight check can run at any time	free, or someone else's	Sync status card; Power enabled when it's safe
checking	Read the cloud manifest, scan the folder, build a plan	unchanged	"Checking for changes..."
claiming	Write a claim, wait 3 s, read it back	claiming	"Reserving the server..."
downloading	Run the download part of the plan	downloading	Transfers panel
launching	Start playit.gg and Java, wait for "Done (" (with timeouts)	launching	Console and progress line
running	Track changes; refresh the player list	running + players	Changes panel; Stop enabled
stopping	Send `stop`, wait for Java to exit (kill after a timeout)	stopping	"Saving the world..."
uploading	Fresh scan, plan, upload, commit the manifest	uploading	Transfers panel
releasing	Mark the lock free	free	Back to the status card

Here's the core of the controller. It's simplified, but it's real TypeScript, and the shape is the point: **one method per action, each guarded by the current phase, each step awaited, and every failure handled in one place.**

ELECTRON/SESSION/SESSIONCONTROLLER.TS (SIMPLIFIED)

```
export class SessionController {
  private state: SessionState = { phase: 'idle' };

  constructor(
    private lock: LockService, private sync: SyncEngine,
    private minecraft: MinecraftProcess, private playit: PlayitProcess,
    private emit: (state: SessionState) => void,  // sends 'session:state' to the UI
  ) {
    minecraft.onExit(() => this.stop());          // a crash takes the normal stop path
  }

  async start(serverId: string) {
    if (this.state.phase !== 'idle') throw new AppError('BUSY', 'Already running.');
    try {
      this.set({ phase: 'checking', serverId });
      const report = await this.sync.preflight(serverId);
      if (report.status === 'diverged') throw new AppError('CONFLICT', 'Pick a version.');

      this.set({ phase: 'claiming', serverId });
      await this.lock.claim(serverId);            // renews every 15 s from here on

      this.set({ phase: 'downloading', serverId });
      await this.sync.download(serverId, report);  // transfers, then saves the new base

      this.set({ phase: 'launching', serverId });
      const address = await this.playit.start({ timeoutMs: 60_000 });
      await this.minecraft.start({ timeoutMs: 180_000 });  // resolves on "Done ("

      this.set({ phase: 'running', serverId, address });
    } catch (err) {
      await this.minecraft.stopAndWait();         // harmless if it never started
      await this.playit.stop();
      await this.lock.releaseIfMine(serverId);    // never touches someone else's lock
      this.set({ phase: 'failed', serverId, error: toAppError(err) });
      throw err;
    }
  }

  async stop() {
    if (this.state.phase !== 'running') return;    // a second call does nothing
    const { serverId } = this.state;
    this.set({ phase: 'stopping', serverId });
    await this.minecraft.stopAndWait({ timeoutMs: 60_000 });  // waits for the exit
    await this.playit.stop();

    this.set({ phase: 'uploading', serverId });
    try {
      await this.sync.uploadSession(serverId);    // scan → plan → transfer → commit
    } catch (err) {
      // The lock stays ours; the Retry button calls retryUpload().
      this.set({ phase: 'upload-pending', serverId, error: toAppError(err) });
      return;
    }
    this.set({ phase: 'releasing', serverId });
    await this.lock.release(serverId);
    this.set({ phase: 'idle' });
  }
```

```
  private set(next: SessionState) { this.state = next; this.emit(next); }
}
```

WHAT THIS FIXES FROM PART 1

The upload can't start before Java has exited (F5). Stop runs once, however many times it's called (F9). Every wait has a timeout and every failure releases what it holds (F21). The shutdown sequence exists once instead of three times (F30). Quitting the app and logging out just call `stop()` and wait for the phase to become idle.

The phases themselves are a type, so the compiler knows which fields exist in which phase:

SHARED/TYPES.TS (EXCERPT)

```
export type SessionState =
  | { phase: 'idle' }
  | { phase: 'checking' | 'claiming' | 'downloading' | 'launching'
           | 'stopping' | 'uploading' | 'releasing'; serverId: string }
  | { phase: 'running'; serverId: string; address?: string }
  | { phase: 'failed' | 'upload-pending'; serverId: string; error: AppError };
```

05 A lock you can trust

Keep lock.json; change how it's used. The new version records *which session* holds it, not just which person:

.LAUNCHER/LOCK.JSON ON DRIVE

```
{
  "format": 2,
  "sessionId": "5f0c2a7e-4b1d-4c55-9a3e-0d6b1f2c9e81",
  "host": { "name": "Steve", "email": "steve@example.com" },
  "phase": "running",
  "leaseSeconds": 45,
  "baseRevision": 42,
  "address": "abc-def.joinmc.link",
  "players": ["Steve", "Alex"]
}
```

A **lease** is a lock with a timer: it's yours only as long as you keep renewing it, so a crashed PC can't hold the server forever. Six rules make it trustworthy:

- **Readers never write.** `inspect()` only reads and answers *free*, *mine*, *held* or *stale* (held but expired). Looking at a server can't change it (F11).
- **Google's clock is the judge.** Expiry is measured from the file's `modifiedTime`, compared with the `Date` header of the same response. Both come from Google, so a PC with a wrong clock can't fool anyone (F34).
- **Claim, then verify.** Write your claim, wait three seconds, read it back. It's yours only if the session ID is yours.

- **Renew for the whole job**, every 15 seconds from the claim to the release, downloads and uploads included (F6). With a 45-second lease, two missed renewals in a row are survivable.
- **Every renewal re-checks ownership.** If the session ID isn't yours any more, the session stops and tells you why.
- **A stale lock that says "uploading" isn't free.** It means someone's upload never finished, so the app explains that instead of letting you start on a half-uploaded world.

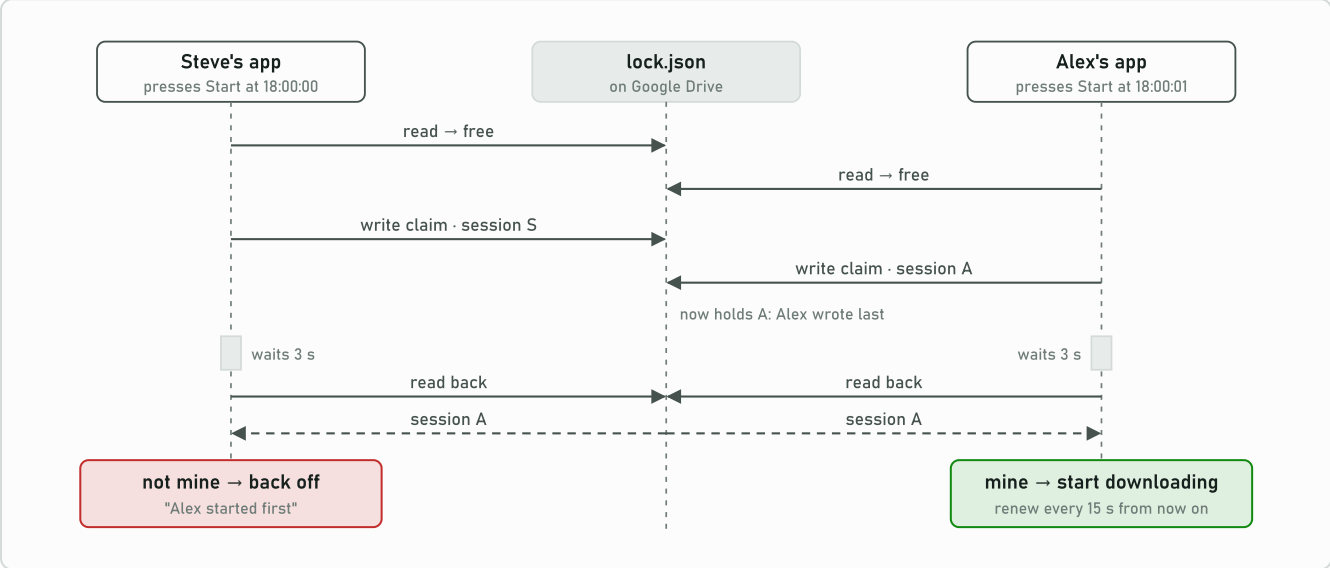


Figure 4. Two members press Start one second apart. Both see a free lock and both write a claim, but only one claim can be in the file afterwards, and reading it back tells each app who won.

ELECTRON/LOCK/LOCKSERVICE.TS (SIMPLIFIED)

```
const LEASE_SECONDS = 45;
const RENEW_EVERY_MS = 15_000;

export class LockService {
  private sessionId = crypto.randomUUID();       // new for every session
  private phase: LockPhase = 'claiming';
  private timer?: NodeJS.Timeout;

  constructor(
    private drive: DriveClient, private me: Host,
    private onLost: (reason: string) => void,
  ) {}

  /** Read-only. Never writes, whatever it finds. */
  async inspect(serverId: string): Promise<LockView> {
    // the content, the file's modifiedTime and the response's Date header
    const file = await this.drive.readLock(serverId);
    if (!file || file.content.phase === 'free') return { state: 'free' };
    const lock = file.content;
    if (lock.sessionId === this.sessionId) return { state: 'mine', lock };
    const ageSeconds = (file.serverNow - file.modifiedTime) / 1000;  // Google's clock
    return { state: ageSeconds > lock.leaseSeconds ? 'stale' : 'held', lock };
  }

  async claim(serverId: string) {
    const before = await this.inspect(serverId);
    if (before.state === 'held')
      throw new AppError('LOCK_HELD', `${before.lock.host.name} is using the server.`);
    if (before.state === 'stale' && before.lock.phase === 'uploading')
      throw new AppError('UNFINISHED_UPLOAD', 'The last upload never finished.');

    await this.write(serverId, 'claiming');
    await sleep(3_000);                          // give a competing claim time to land
    if ((await this.inspect(serverId)).state !== 'mine')
      throw new AppError('LOCK_HELD', 'Someone else started at the same moment.');
    this.scheduleRenewal(serverId);
  }

  // A setTimeout chain instead of setInterval, so two renewals never overlap.
  private scheduleRenewal(serverId: string) {
    this.timer = setTimeout(async () => {
      try {
        if ((await this.inspect(serverId)).state !== 'mine')
          return this.onLost('Another member took over the server.');
        await this.write(serverId, this.phase);  // same content, fresh modifiedTime
      } catch {
        // A network blip. The 45 s lease survives two missed renewals: try again.
      }
      this.scheduleRenewal(serverId);
    }, RENEW_EVERY_MS);
  }
}
```

AN HONEST LIMIT

Google Drive doesn't offer a "write this only if nobody changed it since I read it" operation, which is what a perfect lock needs. Claim-then-verify shrinks the window for a tie from minutes to a couple of seconds, and the ownership check on every renewal is the second safety net: if two apps ever both believed they won, the loser finds out within 15 seconds, long before its server finishes starting. For a group of friends that's plenty.

06 Knowing what changed: three snapshots

Part 1 showed that one version number per side can't tell "I changed this" from "they changed this". The fix is the same idea git uses. Keep a third snapshot, the **base**: the state your PC and the cloud last agreed on. Then compare file by file.

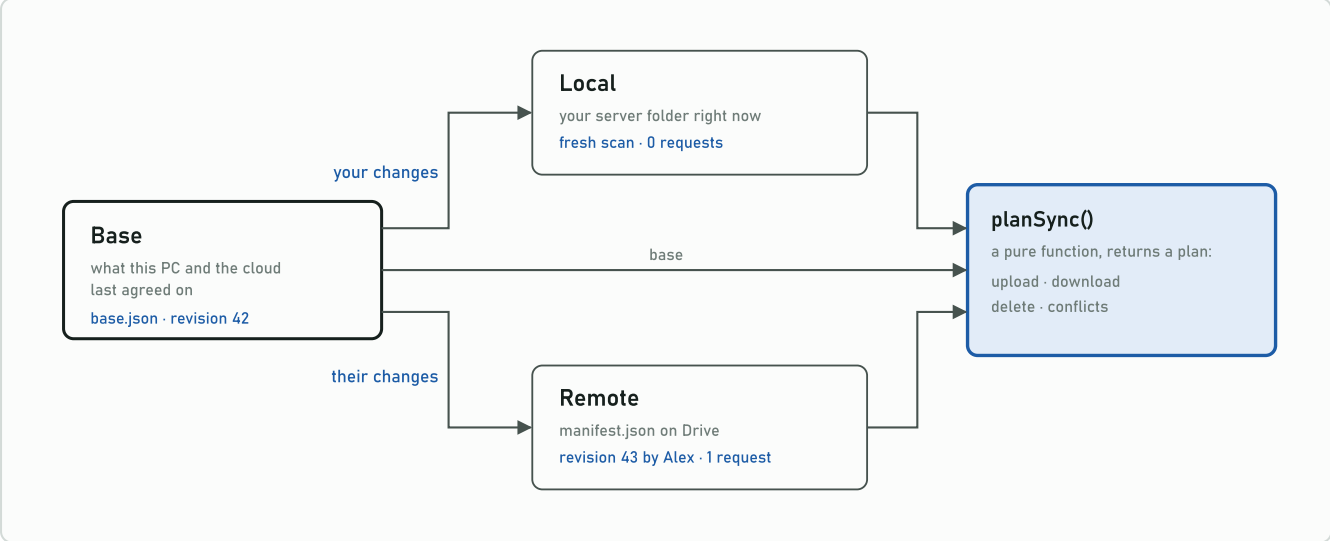


Figure 5. Base versus local gives your changes; base versus remote gives theirs. A file that changed on only one side is safe to copy across. A file that changed on both sides, differently, is a conflict.

SNAPSHOT	WHERE IT COMES FROM	COST	WHEN IT'S WRITTEN
Remote	.launcher/manifest.json on Drive	1 request	Last, by every successful upload (the commit)
Base	base.json in the app's data folder, one per server	none	After every successful download or upload
Local	A scan of the server folder, done fresh each time	no requests	Never stored; always computed

The manifest, version 2

Two changes do most of the work. Files are identified by a **content hash** (MD5) instead of a counter, and each entry remembers the file's **Drive ID**, so a file can be fetched or updated directly without listing folders first. A hash is a short fingerprint of a file's bytes: the same bytes always give the same hash, and different bytes almost always give a different one.

.LAUNCHER/MANIFEST.JSON ON DRIVE

```
{
  "format": 2,
  "revision": 43,
  "parentRevision": 42,
  "author": { "name": "Alex", "email": "alex@example.com" },
  "createdAt": "2026-10-14T18:05:12Z",
  "files": {
    "server.properties": {
      "size": 1342, "md5": "9e107d9d372bb6826bd81d3542a419d6", "driveId": "1Ab…"
    },
    "world/level.dat": {
      "size": 4410, "md5": "e4d909c290d0fb1ca068ffaddf22cbd0", "driveId": "1Cd…"
    },
    "world/region/r.0.0.mca": {
      "size": 7340032, "md5": "0cc175b9c0f1b6a831c399e269772661", "driveId": "1Ef…"
    }
  }
}
```

- **revision / parentRevision.** Each upload says which revision it started from. Before committing, the app checks that the cloud is still at `parentRevision`; if not, someone uploaded in between and the commit stops instead of overwriting their work.
- **Keys are always forward-slash paths** relative to the server folder, produced by one `toPosix()` function (F23, F25).
- **md5** matches the `md5Checksum` field Google stores for every uploaded file, so after an upload the app can compare its own hash with Google's and know the bytes arrived intact.
- **The app's own files live in a `.launcher` folder** on Drive, away from the server files, so there's nothing to blacklist.

The base is the same format plus one field per file, `mtimeMs`: the modified time *on this PC* right after the last sync. That's what makes the scan fast.

The scan: hash only what looks different

Hashing 10,000 files on every check would be slow. Git solved this long ago: if a file's size and modified time are exactly what they were right after the last sync, assume it's unchanged and reuse the stored hash. Only files that look different get hashed. A Minecraft session touches a few hundred files at most, so a typical check stats 10,000 files and hashes a few hundred, without a single network request.

ELECTRON/SYNC/SCANNER.TS

```
export async function scanLocal(
  root: string, base: Snapshot, rules: SyncRules,
): Promise<Snapshot> {
  const local: Snapshot = new Map();
  for await (const file of walk(root)) {         // yields { relPath, size, mtimeMs }
    const path = toPosix(file.relPath);          // "world/region/r.0.0.mca" on every OS
    if (rules.isIgnored(path)) continue;
    const known = base.get(path);
    const looksSame = known && known.size === file.size && known.mtimeMs === file.mtimeMs;
    local.set(path, {
      size: file.size,
      mtimeMs: file.mtimeMs,
      md5: looksSame ? known.md5 : await md5OfFile(join(root, file.relPath)),
    });
  }
  return local;
}

async function md5OfFile(file: string): Promise<string> {
  const hash = createHash('md5');
  for await (const chunk of createReadStream(file)) hash.update(chunk);  // streamed
  return hash.digest('hex');
}
```

This also fixes "downloaded files look modified" (F16): after a download the app writes the new base using the *local* modified time it just observed, so the next scan sees an exact match.

The planner: a pure function

A **pure function** only looks at its arguments and only produces its return value: no disk, no network, no global variables. Give it the same three snapshots and it always returns the same plan. That makes it the easiest code in the app to test, and it's the code you most need to be right.

YOUR COPY VS BASE	CLOUD VS BASE	PLAN
same	same	nothing to do
changed or added	same	**upload** (after your session)
deleted	same	**delete in the cloud**
same	changed or added	**download**
same	deleted	**delete locally**
changed	changed to the same bytes	nothing to do
changed	changed differently	**conflict**

ELECTRON/SYNC/PLANNER.TS

```
export function planSync(base: Snapshot, local: Snapshot, remote: Snapshot): SyncPlan {
  const plan: SyncPlan = {
    upload: [], download: [], deleteRemote: [], deleteLocal: [], conflicts: [],
  };
  const paths = new Set([...base.keys(), ...local.keys(), ...remote.keys()]);

  for (const path of paths) {
    // undefined means "no such file on that side"
    const b = base.get(path)?.md5;
    const l = local.get(path)?.md5;
    const r = remote.get(path)?.md5;

    if (l === b && r === b) continue;              // nobody touched it
    if (l === r) continue;                         // both made the same change
    if (l !== b && r !== b) { plan.conflicts.push(path); continue; }

    if (l !== b) (l === undefined ? plan.deleteRemote : plan.upload).push(path);
    else         (r === undefined ? plan.deleteLocal : plan.download).push(path);
  }
  return plan;
}
```

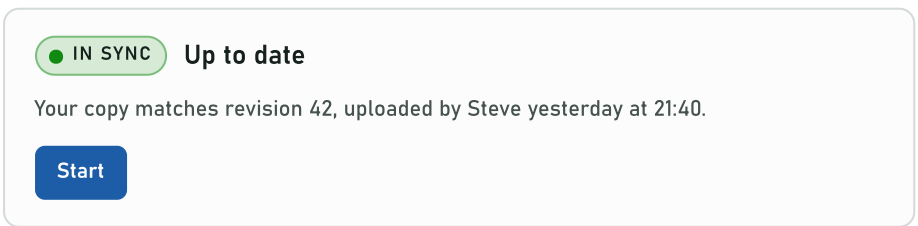
Conflicts: choose a whole world, never a file

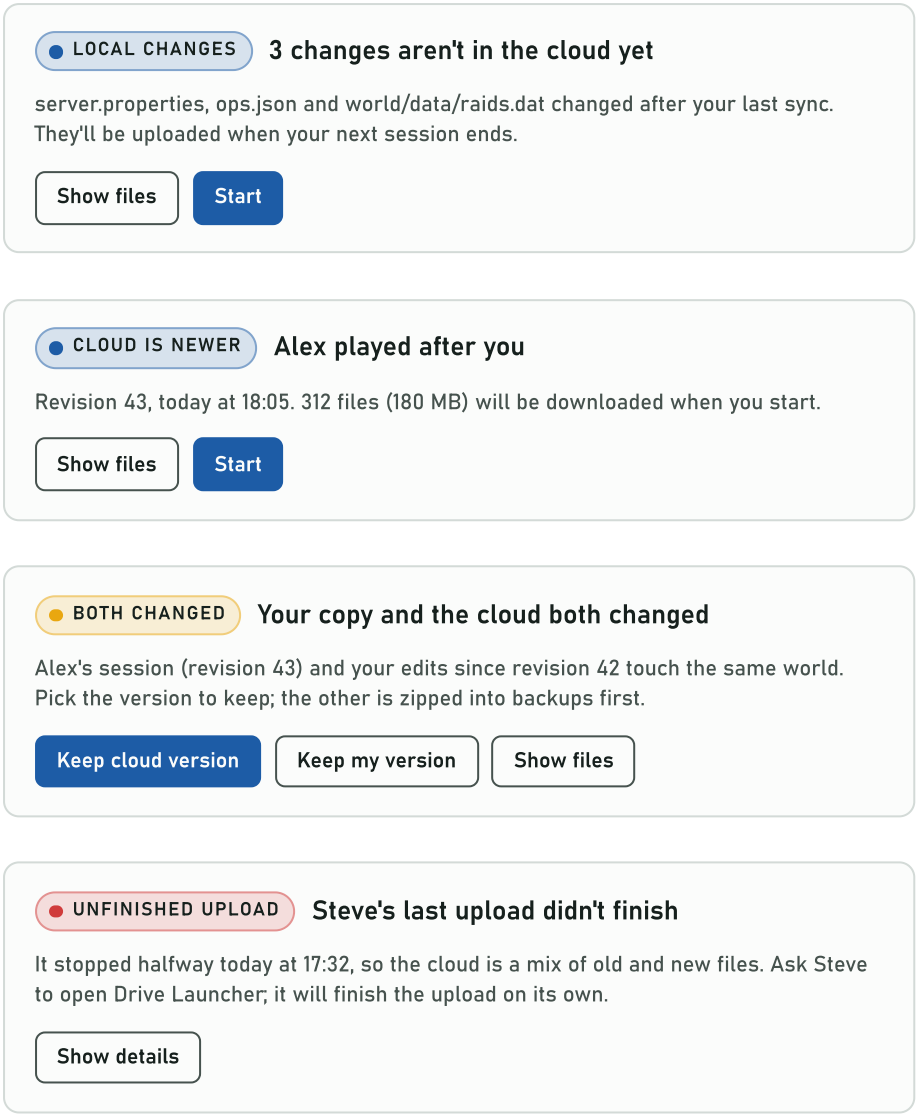
When both sides changed, it's tempting to merge file by file. Don't. A Minecraft world is one connected thing: level.dat, the region files, the entities and the player data all have to come from the same moment, or you get items duplicated, chunks rolled back, or a player standing inside a wall. So a conflict becomes one question for the user, *"Keep the cloud version or keep yours?"*, and before applying the answer the app zips the losing side into the server's backups folder. Nobody loses work silently, and nobody gets a Frankenstein world.

> THE FIRST SYNC ON A NEW PC
>
> A PC that has never synced has no base. If its folder is empty, it simply downloads everything. If it isn't, the app asks once, "Use the cloud version (recommended) or upload this folder?", and backs up whatever it replaces. A brand-new server with no manifest in the cloud yet works the same way in reverse: the first host's folder becomes revision 1.

What the user sees: the sync status card

The preflight check runs when you select a server and again right before Start. Its result replaces today's "is not being hosted right now" line with a card that answers the real question: *is it safe to start, and what will happen if I do?*





07 Uploading safely

An upload changes many files on Drive one at a time, so there's always a moment when some are new and some are old. The design makes that moment harmless with two ideas. The **commit**: manifest.json is uploaded last, and the new revision only exists once it lands. The **journal**: before starting, the app writes the plan to a file on your PC, so a crash can be finished later instead of forgotten.

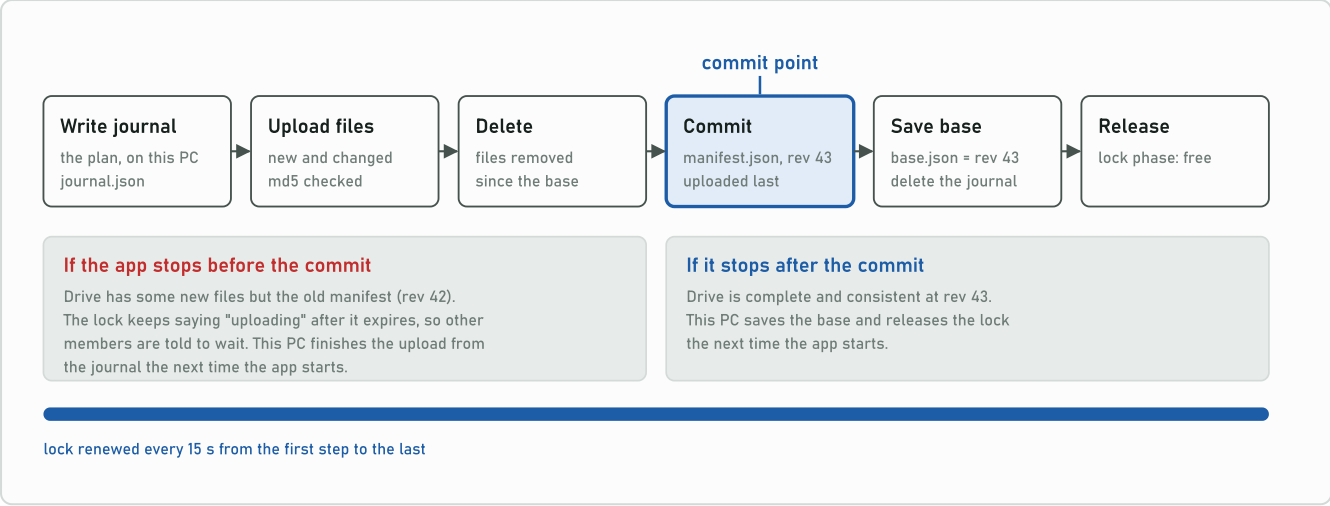


Figure 6. The upload as six steps with one commit point. A crash before it is recoverable from the journal, and a crash after it only leaves local housekeeping. At no point does the app report success for work that hasn't been confirmed.

ELECTRON/SYNC/SYNCENGINE.TS (UPLOADSESSION, SIMPLIFIED)

```
async uploadSession(serverId: string) {
  const base = await this.appData.readBase(serverId);
  // Java has exited by now, so the folder is quiet.
  const local = await scanLocal(this.root(serverId), base, this.rules);
  const plan = planUpload(base, local);                  // pure, like planSync
  if (plan.isEmpty) return;

  await this.appData.writeJournal(serverId, { plan, startedAt: Date.now() });
  const results = await this.transfers.run(plan.jobs);   // lanes, retries, md5 checks
  if (results.failed.length)
    throw new AppError('UPLOAD_INCOMPLETE', `${results.failed.length} files failed.`);

  const remote = await this.drive.readManifest(serverId);
  if (remote.revision !== base.revision)                 // someone committed meanwhile
    throw new AppError('CONFLICT', 'The cloud changed during your upload.');
  const next = buildManifest(base, results, {
    revision: base.revision + 1,
    parentRevision: base.revision,
  });
  await this.drive.writeManifest(serverId, next);        // ← the commit

  await this.appData.writeBase(serverId, toBase(next, local));
  await this.appData.deleteJournal(serverId);
}
```

An advanced alternative, for later: never overwrite a file on Drive. Upload each new version as a new file, let the manifest point to the new IDs, and delete the old ones after the commit. Then the commit really is all-or-nothing, at the cost of briefly storing two copies of the changed files.

08 The transfer engine

Before any code, a word about "threads". Node runs your JavaScript on one thread, and that's fine here: a transfer spends almost all of its time *waiting* for Google. Running six transfers at once means six requests waiting at the same time, which is pure gain. These are called **lanes** below,

because each one carries one file at a time, and Bottleneck is the traffic controller that decides how many lanes are open.

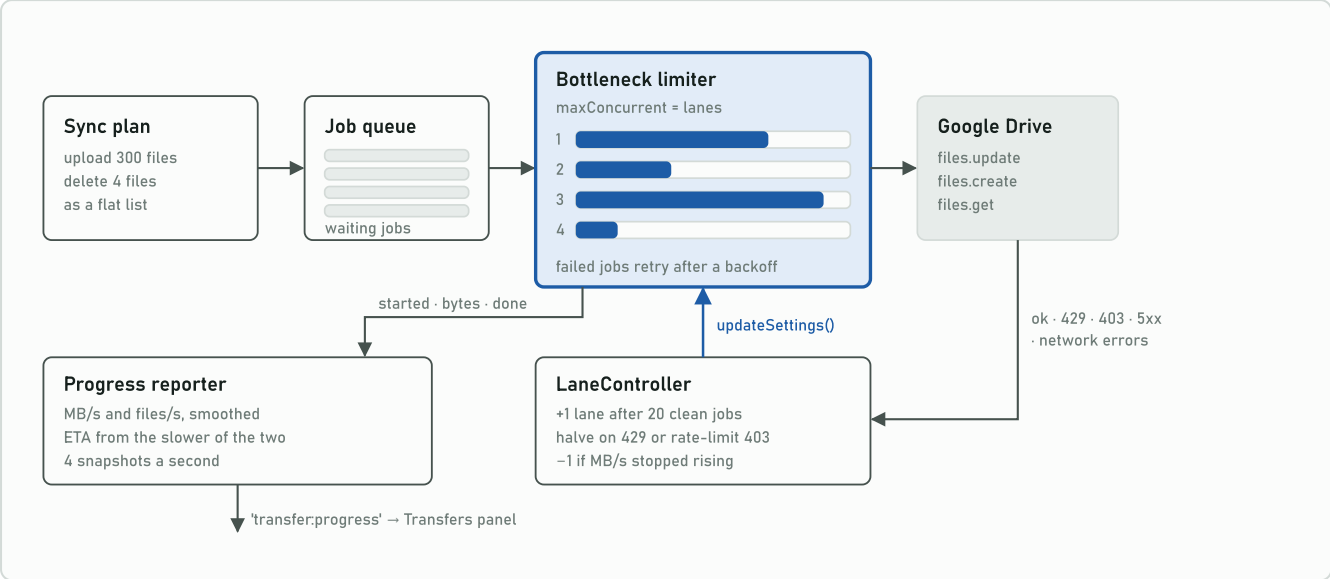


Figure 7. The engine. The plan becomes a flat queue, Bottleneck runs as many jobs at once as there are lanes, Google's answers adjust the lane count, and job events become progress snapshots for the UI.

Folders first, then a flat list of files

Part 1's walkers listed every Drive folder on every run (F19) and lost the full path on the way down (F4). The new engine never walks a tree. Every file already has a Drive ID in the manifest, so the only folder work left is creating folders that don't exist yet. That happens first, parents before children, and each new folder's ID is remembered in a `Map` from path to ID. After that, every file is an independent job, which is exactly what makes parallel lanes possible.

One request per file where possible

JOB	REQUEST	REQUESTS	NOTES
Download	`GET files/{driveId}?alt=media`	1	Streamed to `name.part`, hash checked, then renamed into place
Update, ≤ 5 MB	`PATCH upload/files/{driveId}?uploadType=media`	1	Most changed files: playerdata, stats, level.dat
New file, ≤ 5 MB	`POST upload/files?uploadType=multipart`	1	Name, parent folder and bytes in one request
Any file > 5 MB	`uploadType=resumable`, sent in chunks	2+	Large region files; can resume after a dropped connection
Delete	`DELETE files/{driveId}`	1	Or move to Drive's trash, if you prefer an undo

Uploads ask Google to return `id,md5Checksum,size`. If Google's checksum doesn't match the one the app computed, the job fails and retries, so a corrupted transfer can never be committed. The 5

MB line is Google's own guidance: simple and multipart uploads for files of 5 MB or less, resumable uploads above that.

How many lanes? Add slowly, back off fast

There's no single right number. It depends on your upload speed, Google's mood and how many small files there are. So the engine measures and adjusts, using the rule TCP itself uses on the internet, called **AIMD** (additive increase, multiplicative decrease): add one lane at a time while things go well, and cut the lanes in half the moment Google says "too many requests".

SIGNAL	REACTION
20 jobs in a row finished without throttling	Open one more lane, up to 12
Google answered 429, or 403 with reason `rateLimitExceeded` / `userRateLimitExceeded`	Halve the lanes (at least 1) and retry that job after a backoff
A lane was added, but 10 s later MB/s rose by less than 5%	Close that lane again and stop adding: your connection is full
Network error or a 5xx answer	Retry that job after a backoff; lanes unchanged
The same job failed 5 times	Mark it failed and keep going with the others

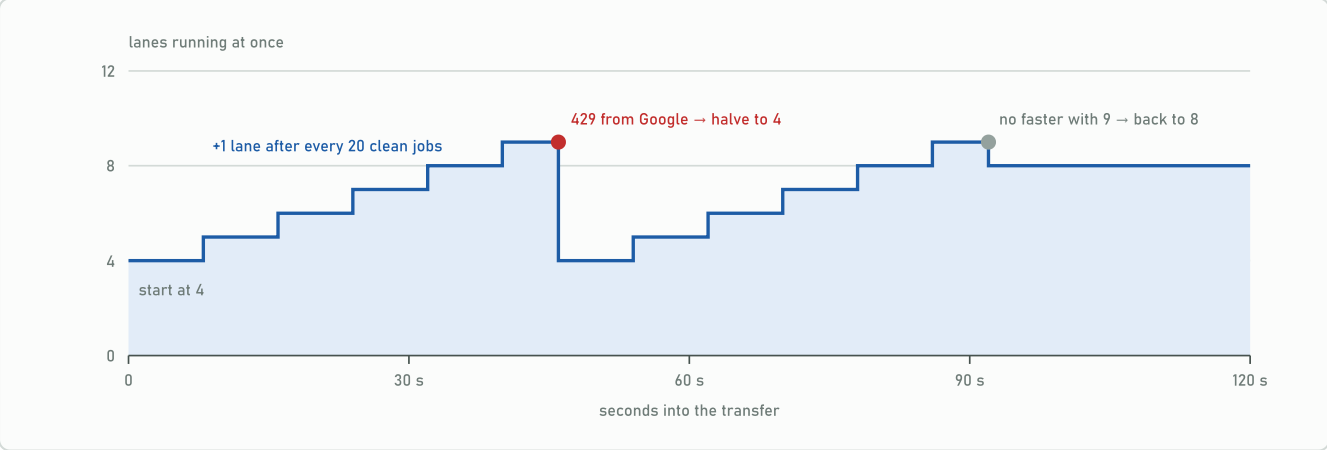


Figure 8. An example of the lane count during a two-minute upload: a slow climb, a sharp cut when Google pushes back, and a plateau once the connection is full. The sawtooth shape is what AIMD looks like.

ELECTRON/TRANSFER/TRANSFERENGINE.TS (THE CORE, SIMPLIFIED)

```
import Bottleneck from 'bottleneck';

export class TransferEngine {
  private limiter = new Bottleneck({ maxConcurrent: 4 });
  private lanes = new LaneController({ start: 4, min: 1, max: 12 });

  constructor(private drive: DriveClient, private progress: ProgressReporter) {
    this.lanes.onChange((n) => this.limiter.updateSettings({ maxConcurrent: n }));
    this.limiter.on('failed', (error, info) => {
      // 429, or 403 with a rate-limit reason
      if (isRateLimit(error)) this.lanes.onThrottled();
      // Returning a number means "retry after that many ms".
      // Returning nothing means "give up": the job's promise rejects.
      if (isRetryable(error) && info.retryCount < 5) return backoffMs(info.retryCount);
    });
  }

  async run(jobs: TransferJob[]): Promise<TransferResult> {
    const result: TransferResult = { done: [], failed: [] };
    await Promise.all(jobs.map((job) =>
      this.limiter.schedule(() => this.runJob(job))  // one request, md5 checked
        .then((moved) => {
          this.lanes.onSuccess();
          this.progress.done(job);
          result.done.push(moved);
        })
        .catch((error) => {
          this.progress.failed(job, error);
          result.failed.push({ job, error });
        }),
    ));
    return result;                // the caller sees every failure, never a hidden one
  }
}

// Google's advice: wait 2^n seconds plus up to 1 s of random "jitter", with a cap.
const backoffMs = (attempt: number) =>
  Math.min(2 ** attempt * 1000 + Math.random() * 1000, 32_000);
```

ELECTRON/TRANSFER/LANECONTROLLER.TS

```
export class LaneController {
  private lanes: number;
  private ceiling: number;
  private cleanStreak = 0;
  private listeners: Array<(lanes: number) => void> = [];

  constructor(private opts: { start: number; min: number; max: number }) {
    this.lanes = opts.start;
    this.ceiling = opts.max;
  }

  onSuccess() {                                  // additive increase
    if (++this.cleanStreak < 20 || this.lanes >= this.ceiling) return;
    this.cleanStreak = 0;
    this.set(this.lanes + 1);
  }
  onThrottled() {                                // multiplicative decrease
    this.cleanStreak = 0;
    this.set(Math.max(this.opts.min, Math.floor(this.lanes / 2)));
  }
  onNoSpeedGain() {                              // the connection is full
    this.ceiling = Math.max(this.opts.min, this.lanes - 1);
    this.set(this.ceiling);
  }

  onChange(listener: (lanes: number) => void) { this.listeners.push(listener); }
  private set(lanes: number) {
    if (lanes === this.lanes) return;
    this.lanes = lanes;
    for (const listener of this.listeners) listener(lanes);
  }
}
```

Jitter, the random part of the backoff, matters more than it looks: without it, six lanes that were throttled at the same moment would all retry at the same moment and get throttled again. **Which errors are retryable** is a short list: network errors, 429, 500, 502, 503, 504, and 403 only when its reason is a rate limit. A 401 means the token expired, so DriveClient refreshes it once and repeats the request. Everything else, such as 404 or a 403 for missing permission, fails the job immediately with a message the user can act on.

HOW MUCH CAN GOOGLE TAKE?

Since 1 May 2026 Google measures Drive API usage in quota units: each user may spend 325,000 units per minute in a project, a download costs 200 units, an edit 50 and a list 100. That allows roughly 1,600 downloads a minute per user, far more than one PC's connection can feed, but Google can still answer 429 when its own servers are busy. That's why the engine reacts to answers instead of trusting a fixed number. Bottleneck itself is old but stable (its latest release, 2.19.5, is several years old); if you ever want an actively maintained alternative, `p-queue` does the same job.

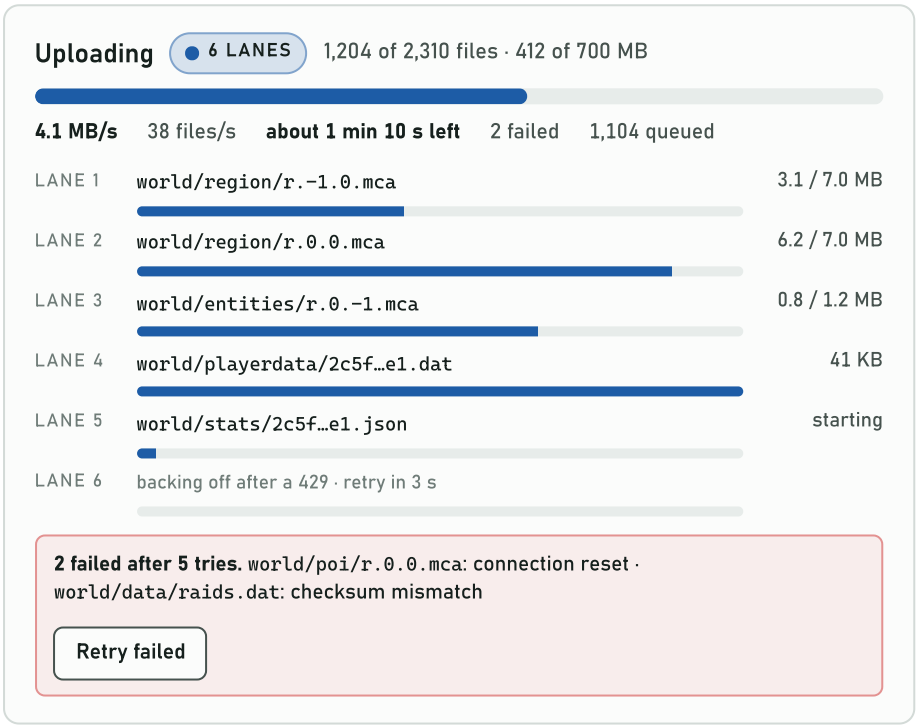
Progress and a time estimate that doesn't jump around

The engine records every job event, and a reporter sends a snapshot to the UI four times a second. Raw speed jumps all over the place, so it's smoothed with an **exponentially weighted moving average** (EWMA): each tick, the new speed is 80% the old speed plus 20% what was just measured. Recent measurements count most, but one slow second doesn't wreck the estimate.

ELECTRON/TRANSFER/PROGRESS.TS (EVERY 250 MS)

```
bytesPerSec = 0.8 * bytesPerSec + 0.2 * (bytesSinceLastTick / 0.25);
filesPerSec = 0.8 * filesPerSec + 0.2 * (filesSinceLastTick / 0.25);

// Big files are limited by bandwidth, thousands of small files by round trips.
// Estimate both ways and trust the slower one.
const etaSec = Math.max(remainingBytes / bytesPerSec, remainingFiles / filesPerSec);
```



The Transfers panel, as an example. Every number on it comes straight from one progress snapshot; the panel doesn't calculate anything itself.

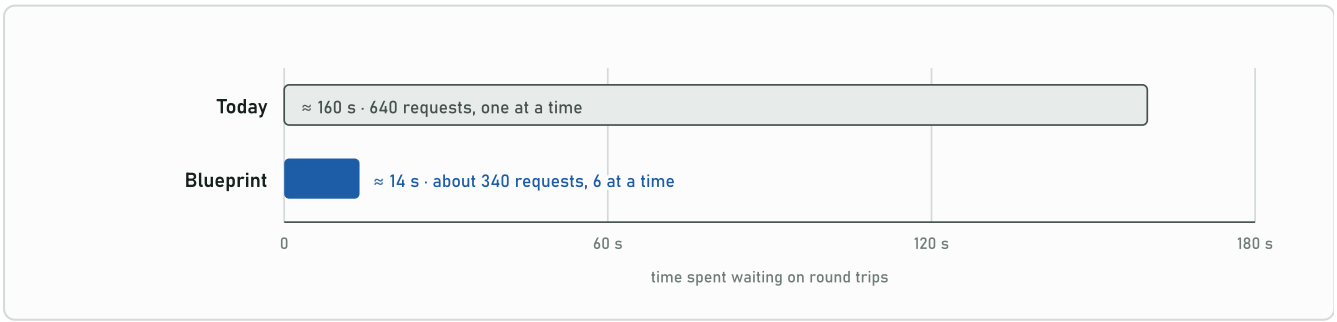


Figure 9. Back-of-the-envelope comparison for the upload from Part 1, section 9: 300 changed files in 40 folders, about 0.25 s per round trip, 6 lanes. The time to move the bytes themselves depends on your connection and is left out of both bars.

09 Watching the session live

The watcher gets a smaller, clearer job: **it powers the live Changes panel and nothing else.** What gets uploaded is decided by a fresh scan after Java exits, so a missed watcher event can never lose data. That one decision removes the hardest problems from Part 1's watcher.

- **It starts when the phase becomes running and stops at stopping.** No manual Start button.
- `ignoreInitial: true`**.** No flood of 10,000 add events at startup (F17); the preflight scan already knows what's there.
- **Events are batched.** Changes are collected for half a second and sent as one message. The renderer keeps them in a Map and groups them by folder.

ELECTRON/SYNC/CHANGETRACKER.TS

```
const pending = new Map<string, 'added' | 'changed' | 'removed'>();
let timer: NodeJS.Timeout | null = null;

function record(absPath: string, kind: 'added' | 'changed' | 'removed') {
  const path = toPosix(relative(root, absPath));
  if (rules.isIgnored(path)) return;
  pending.set(path, kind);          // a newer event for the same file replaces the older
  timer ??= setTimeout(flush, 500);
}

function flush() {
  timer = null;
  // One message for everything since the last flush, for example
  // [["world/region/r.0.0.mca", "changed"], ["world/level.dat", "changed"]]
  emit('changes:batch', [...pending]);
  pending.clear();
}

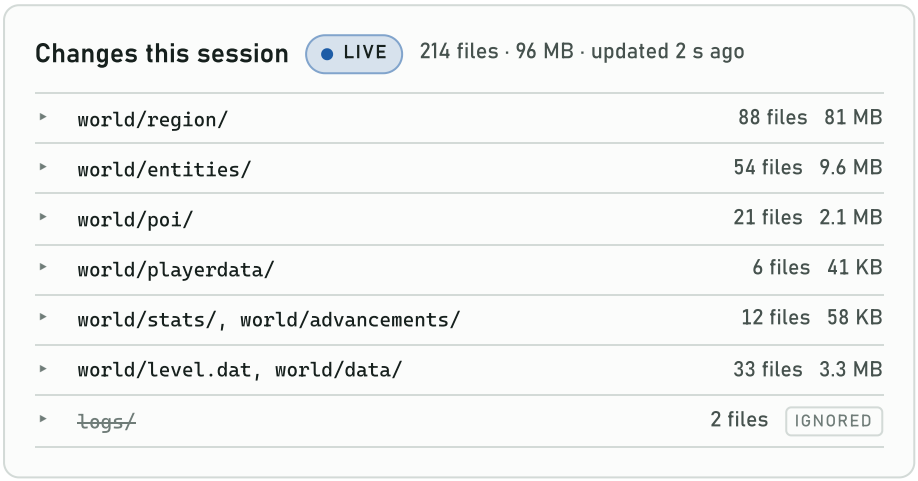
watcher = chokidar
  .watch(root, { ignoreInitial: true, ignored: (p) => rules.isIgnoredAbsolute(p) })
  .on('add', (p) => record(p, 'added'))
  .on('change', (p) => record(p, 'changed'))
  .on('unlink', (p) => record(p, 'removed'));
```

Shared ignore rules

The blacklist becomes `.launcher/sync-rules.json` on Drive, so every member syncs the same set of files (F24). Patterns use forward slashes and three simple forms: a folder (`logs/`), an exact file, or an extension (`*.tmp`). If you want full glob patterns later, the `picomatch` package does that well. Built-in rules live in code and are always added on top (F22).

.LAUNCHER/SYNC-RULES.JSON ON DRIVE

```
{
  "format": 1,
  "ignore": [
    "logs/",
    "crash-reports/",
    "world/session.lock",
    "*.tmp",
    "world/data/DistantHorizons.sqlite"
  ]
}
```



The Changes panel, as an example: grouped by folder so 200 files read as seven lines, with ignored files shown but crossed out, so nobody wonders why their logs didn't upload.

10 The bridge: typed IPC and one error convention

Part 1 counted 43 request channels and 12 events, all plain strings (F31). The new bridge has one contract file. Both processes import it, so a wrong channel name or a wrong argument is a compile error, and the list shrinks to about twenty commands because the session now takes intents instead of individual steps.

SHARED/IPC.TS

```
export type Result<T> =
  | { ok: true; value: T }
  | { ok: false; error: AppError };

export interface AppError {
  code: 'NOT_SIGNED_IN' | 'LOCK_HELD' | 'UNFINISHED_UPLOAD' | 'CONFLICT' | 'BUSY'
      | 'NETWORK' | 'RATE_LIMITED' | 'NOT_FOUND' | 'UPLOAD_INCOMPLETE' | 'UNKNOWN';
  message: string;                          // already written for the user
}

// Every request the renderer can make: name → (arguments) => result
export interface Commands {
  'session:preflight':   (serverId: string) => SyncReport;
  'session:start':       (serverId: string) => void;
  'session:stop':        () => void;
  'session:resolve':     (choice: 'keep-cloud' | 'keep-local') => void;
  'session:retryUpload': () => void;
  'servers:list':        () => ServerSummary[];
  'settings:get':        (serverId: string) => ServerSettings;
  // … about twenty in total
}

// Every event the main process pushes
export interface Events {
  'session:state':     SessionState;
  'transfer:progress': TransferSnapshot;
  'changes:batch':     Array<[path: string, kind: 'added' | 'changed' | 'removed']>;
}
```

ELECTRON/IPC/HANDLE.TS: THE ONLY PLACE WHERE ERRORS BECOME RESULTS

```
export function handle<C extends keyof Commands>(
  channel: C,
  run: (...args: Parameters<Commands[C]>) => Promise<ReturnType<Commands[C]>>,
) {
  ipcMain.handle(channel, async (_event, ...args) => {
    try {
      return { ok: true, value: await run(...(args as Parameters<Commands[C]>)) };
    } catch (err) {
      log.error(channel, err);
      return { ok: false, error: toAppError(err) };
    }
  });
}

// electron/ipc/handlers.ts: one line per channel
handle('session:preflight', (id) => session.preflight(id));
handle('session:start',     (id) => session.start(id));
handle('session:stop',      ()   => session.stop());
```

ELECTRON/PRELOAD.TS AND SRC/API.TS

```
// preload.ts: generic, never needs editing again
contextBridge.exposeInMainWorld('api', {
  invoke: (channel: string, ...args: unknown[]) => ipcRenderer.invoke(channel, ...args),
  on: (channel: string, listener: (payload: unknown) => void) => {
    const wrapped = (_e: unknown, payload: unknown) => listener(payload);
    ipcRenderer.on(channel, wrapped);
    return () => ipcRenderer.off(channel, wrapped);   // its own unsubscribe (F37)
  },
});

// api.ts: the typed face of window.api
export const invoke = <C extends keyof Commands>(
  channel: C, ...args: Parameters<Commands[C]>
) => window.api.invoke(channel, ...args) as Promise<Result<ReturnType<Commands[C]>>>;

export const on = <E extends keyof Events>(
  event: E, listener: (payload: Events[E]) => void,
) => window.api.on(event, listener as (payload: unknown) => void);

// in a component
const res = await invoke('session:start', server.id);  // a typo here won't compile
if (!res.ok) showError(res.error.message);
```

The renderer's session store becomes a **mirror**: it holds whatever the main process last sent and never changes it on its own. Buttons derive what they allow from the phase, so they can't disagree with the main process.

SRC/STORES/SESSIONSTORE.TS

```
export const useSession = create<{ state: SessionState }>(() => ({
  state: { phase: 'idle' },
}));
on('session:state', (state) => useSession.setState({ state }));

// in the Power button
const phase = useSession((s) => s.state.phase);
const canStart = phase === 'idle';
const canStop = phase === 'running';
```

11 Where every fact lives now

This is Part 1's "one fact, many homes" table again, with one home each.

FACT	ITS ONE OWNER	HOW EVERYONE ELSE SEES IT
Session phase and details	SessionController	`session:state` events; the renderer mirrors them
Who holds the server	lock.json, written only by the holder's LockService	`inspect()`, which never writes
What the cloud contains	.launcher/manifest.json, written only by a commit	Read once per preflight and before each commit
What this PC last synced	base.json in the app's data folder	Read by SyncEngine only
What's on disk now	The disk itself	A fresh scan whenever it matters
Changes during this session	ChangeTracker	`changes:batch` events
Transfer progress	TransferEngine	`transfer:progress` snapshots
Ignored files	.launcher/sync-rules.json on Drive	Loaded at the start of each session
Local settings	Per-server settings.json in the app's data folder	`settings:get` / `settings:set`

The app's data folder gets one subfolder per server, so nothing leaks from one server to another (F14), and the server folder itself contains only server files:

```
%APPDATA%/Drive Launcher/
  token.json
  settings.json               app-wide: window, last selected server
  servers/
    1x9Kq…/                   one folder per server, named by its Drive folder ID
      settings.json           local path, RAM, playit.gg
      base.json               the base snapshot
      journal.json            exists only while an upload is unfinished
      backups/                zips made before resolving a conflict
```

Figure 10. Local data, one folder per server.

12 Testing the parts that decide

You don't need to test everything. Test the code that *decides*, because a wrong decision there loses worlds, and that code is pure, so tests are short. Good candidates: `planSync`, `toPosix` and the ignore rules, `LaneController`, the ETA maths, and LockService's free/mine/held/stale rules. Vitest fits a Vite project with no extra setup.

ELECTRON/SYNC/PLANNER.TEST.TS

```
import { test, expect } from 'vitest';
import { planSync } from './planner';

// Builds a snapshot from { path: md5 }
const snap = (files: Record<string, string>) =>
  new Map(Object.entries(files).map(([path, md5]) => [path, { md5, size: 0, mtimeMs: 0 }]));

test('a file changed only in the cloud is downloaded', () => {
  const base = snap({ 'level.dat': 'a' });
  const plan = planSync(base, snap({ 'level.dat': 'a' }), snap({ 'level.dat': 'b' }));
  expect(plan.download).toEqual(['level.dat']);
});

test('a file changed on both sides is a conflict', () => {
  const base = snap({ 'ops.json': 'a' });
  const plan = planSync(base, snap({ 'ops.json': 'b' }), snap({ 'ops.json': 'c' }));
  expect(plan.conflicts).toEqual(['ops.json']);
});

test('a file deleted locally is deleted in the cloud', () => {
  const base = snap({ 'old.txt': 'a' });
  const plan = planSync(base, snap({}), snap({ 'old.txt': 'a' }));
  expect(plan.deleteRemote).toEqual(['old.txt']);
});
```

(Note the `Object.entries` here: it's called on a plain object, which is what it's for. Part 1's F1 called it on a Map.)

For code that talks to Drive, the trick is **dependency injection**, which only means passing a dependency in instead of creating it inside. TransferEngine receives a DriveClient in its constructor, so a test can hand it a fake that keeps files in a Map and fails on request: "the third upload answers 429". Then you can check that the lanes halve and the job is retried, without touching the internet.

Finally, add two scripts to package.json, `"typecheck": "tsc --noEmit"` and `"test": "vitest run"`, and run both before every commit. That habit alone would have caught F1 and F20.

13 Questions you might have

Why not compare every file with Drive at the end, like the first version did?

Because asking Drive about 10,000 files is 10,000 requests. With the manifest, learning the cloud's whole state costs one request, and learning your own costs zero, since it's a local scan. That's the idea you had; it just needed the base to be complete.

Why hashes, when modified times seemed to work?

Modified times differ between machines and change when a file is downloaded, so they can't be compared across PCs. Hashes describe content, which is what you care about. Modified times are still used, but only on one PC, as a shortcut to avoid re-hashing.

Why not real threads for the transfers?

The work is waiting for the network, not computing. Concurrent requests on one thread handle that perfectly. Worker threads would add message passing and shared-state problems for no speed gain.

Isn't it a problem that Drive can't do atomic writes?

It's why the design has several layers instead of one: claim-then-verify narrows races, renewals re-check ownership, the commit makes revisions all-or-nothing from a reader's point of view, checksums catch damaged or half-updated files, and the journal lets an interrupted upload finish. None is perfect alone; together they're more than a group of friends will ever need.

Do I have to rewrite everything at once?

No, and you shouldn't. The next section is ordered so the app keeps working after every step.

14 Getting there from here

Each step leaves a working app, and you can pause after any of them. The first step alone stops the data loss.

1 Make it build, then stop the data loss

Finish the rename, run `npm install chokidar` so package.json records it, and add the typecheck script. Then the smallest fixes that protect worlds: iterate the Map properly (F1), key entries by full path (F4), upload manifest.json after the files (F2), wait for Java to exit before uploading (F5), keep the heartbeat running through transfers (F6), allow Start only when the lock is offline or expired (F7), guard the stop sequence with a flag (F9), and hide manual Download and Upload while anyone holds the lock (F10). *Check:* host a session, build something, stop, and confirm on drive.google.com that the region files have new modified times.

2 Add the contract and one Drive client

Create shared/ipc.ts, `handle()` and src/api.ts, then move channels over one feature at a time; old string channels keep working meanwhile. Create DriveClient with one OAuth client, a single in-flight token refresh, and retries with backoff. Fixes F26, F28, F31, F36. *Check:* rename a channel in the contract and watch both processes fail to compile.

3 Manifest v2, the scanner and the planner

Write planner.ts with its tests first, then scanner.ts, then the preflight and the sync status card. There's nothing worth migrating from the old manifest: treat the first v2 upload as a fresh start, made by whoever has the newest world. Fixes F3, F13, F14, F15, F16, F25.

4 The transfer engine

Add Bottleneck with `npm install bottleneck`, build TransferEngine and LaneController, switch to flat job lists with Drive IDs, streaming downloads to .part files, one-request uploads for small files, and checksum checks. Then build the Transfers panel. Fixes F12, F18, F19, F35.

5 SessionController and the new lock

Move hosting out of main.ts into the state machine. Build LockService with session IDs, claim-then-verify, the renewal chain and read-only inspection, plus the upload journal and resume. Fixes F5, F6, F7, F8, F9, F11, F21, F27, F30 and F34 properly.

6 Live changes and shared rules

The ChangeTracker with batching, the Changes panel, and sync-rules.json on Drive with an editor that replaces the Blacklist modal. Fixes F17, F22, F23, F24, F39.

G Glossary

TERM	MEANING IN THIS DOCUMENT
AIMD	Additive increase, multiplicative decrease: add one lane at a time, halve on throttling.
Backoff, jitter	Waiting longer after each failure (backoff), plus a random extra (jitter) so retries don't all collide.
Base	The snapshot this PC and the cloud last agreed on. Git calls the same idea the merge base.
Commit	The single write that makes a new revision real: uploading manifest.json, last.
EWMA	Exponentially weighted moving average: a smoothed value where recent samples count most.
Hash, checksum	A short fingerprint of a file's bytes; same bytes, same hash. Drive stores an MD5 for every file.
Idempotent	Doing it twice has the same effect as doing it once, like `stop()` here.
Journal	A note written before a risky job, describing it, so an interrupted job can be finished later.
Lane	One transfer slot. Six lanes means up to six files moving at once.
Lease	A lock with a timer that the holder must keep renewing.
Pure function	Uses only its arguments and only returns a value: no disk, network or globals.
Race condition	A bug that depends on which of two things happens first.
Result type	Returning `{ ok, value }` or `{ ok: false, error }` instead of throwing across a boundary.
State machine	Something that's always in exactly one named state and only moves along allowed transitions.
Streaming	Processing a file in chunks as it flows, instead of loading all of it into memory.

Sources for the Google Drive details: Drive API usage limits (developers.google.com/workspace/drive/api/guides/limits), upload types (…/guides/manage-uploads) and the File resource's checksum fields (…/reference/rest/v3/files). Bottleneck's retry and settings API is from its README.